

NUCLEO-G474RE

Power Electronics Project Series

Project NN

Your Project Title

One-sentence description of what this project demonstrates.

Implementation Frameworks:

STM32CubeIDE | MATLAB Embedded Coder

Document Version: 1.0

Last Updated: June 12, 2026

GitHub: <https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>

This work is licensed under the MIT License

Contents

1	Project Overview	3
1.1	Objective	3
1.2	Applications	3
1.3	Key Concepts	3
1.4	Learning Outcomes	3
1.5	Overall Execution Architecture	3
1.6	Development Framework Paths	4
2	STM32CubeIDE Implementation	6
2.1	Prerequisites	6
2.2	Hardware Configuration	7
2.2.1	Pin Configuration	7
2.2.2	Peripheral Configuration	7
2.3	Software Architecture	7
2.3.1	Project Structure	7
2.3.2	Initialisation Flow	8
2.4	Code Implementation	8
2.4.1	Peripheral Initialisation	8
2.4.2	Core Algorithm / Control Law	8
2.5	Building and Flashing	8
3	MATLAB Embedded Coder Implementation	9
3.1	Prerequisites	9
3.2	MATLAB/Simulink Model Design	9
3.2.1	Model Architecture	9
3.2.2	Subsystem Summary	9
3.3	Model Configuration	10
3.3.1	Solver Settings	10
3.3.2	Hardware Target Settings	10
3.4	Code Generation and Deployment	10
3.5	MATLAB Functions (Codegen)	10
4	Testing and Validation	10
4.1	Hardware Testing	10
4.2	Model Simulation Testing	11
4.3	Integration Testing	11
5	Results and Analysis	11
5.1	Experimental Results	11
5.2	Key Observations	11
6	Troubleshooting	12

7 Extensions and Enhancements	12
7.1 Possible Improvements	12
7.2 Advanced Topics	12
8 Conclusion	12
A Code Listings	14
B Peripheral Configuration Screenshots	14
C Hardware Documentation	14

1 Project Overview

1.1 Objective

[Guidance: State the project goal in 35 words or fewer. Example — “This project implements closed-loop PI control of a buck converter using MATLAB/Simulink and the STM32G474RE ADC/DAC. The architecture splits plant and controller across the PC and the embedded target.”]

1.2 Applications

[Guidance: List 3–5 real-world domains or use cases where this technique is applied. Be specific; avoid generic statements like “useful in industry.”]

- *Application 1 — describe concisely.*
- *Application 2 — describe concisely.*
- *Application 3 — describe concisely.*

1.3 Key Concepts

- **Power Electronics:** *[Guidance: Name the topology, switching device, or conversion principle involved, e.g. “Buck converter, duty-cycle control, CCM operation.”]*
- **Digital Control Systems:** *[Guidance: Name the control law, discretisation method, or relevant concept, e.g. “PI controller, anti-windup, Tustin discretisation.”]*
- **Embedded Firmware / Model-Based Design:** *[Guidance: Describe the MCU-specific concept demonstrated, e.g. “DMA-driven ADC sampling, HRTIM complementary PWM, or Simulink hardware deployment.”]*

1.4 Learning Outcomes

By completing this project, you will:

- *Outcome 1 — begin with a verb, e.g. “Understand how timer prescaler and ARR set the PWM switching frequency.”*
- *Outcome 2.*
- *Outcome 3.*
- *Outcome 4 (optional).*

1.5 Overall Execution Architecture

The diagram below illustrates the generic execution model shared by all projects in this series. Every project follows this pattern: hardware initialisation, an idle main loop, and interrupt-driven callbacks for time-critical operations.

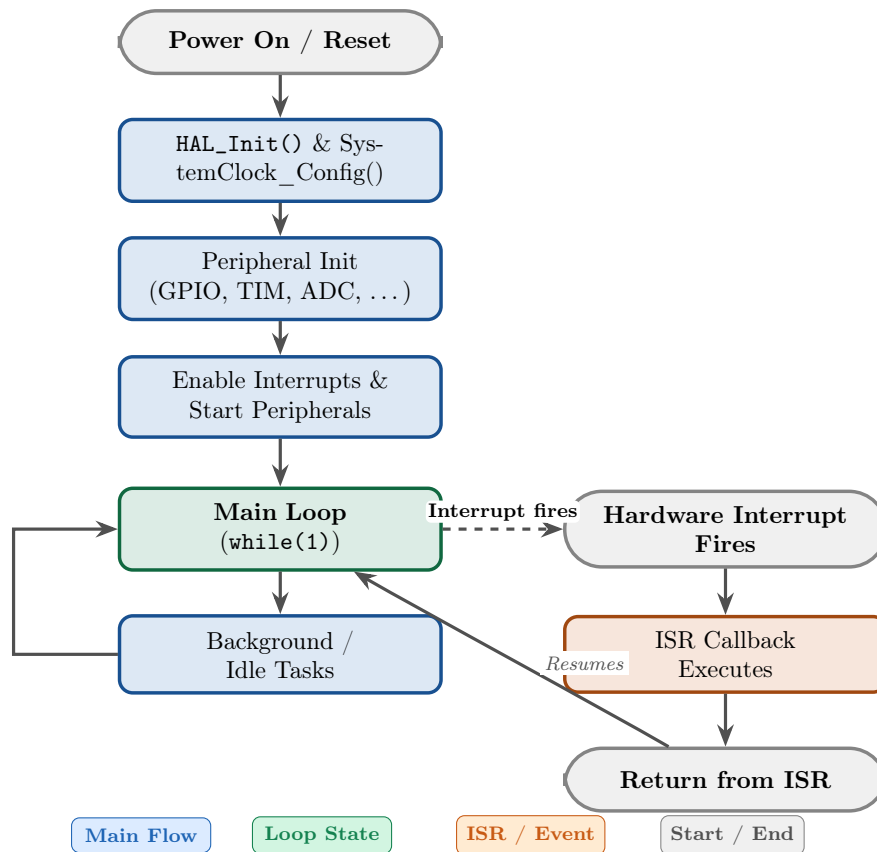


Figure 1: Generic STM32 execution model: hardware initialises once, then the main loop runs indefinitely. Hardware interrupts preempt the loop, execute an ISR callback atomically, then return control. Background tasks run during idle periods between interrupts.

1.6 Development Framework Paths

This series supports two complementary development paths. Figure 2 summarises how each route reaches the same validated embedded target.

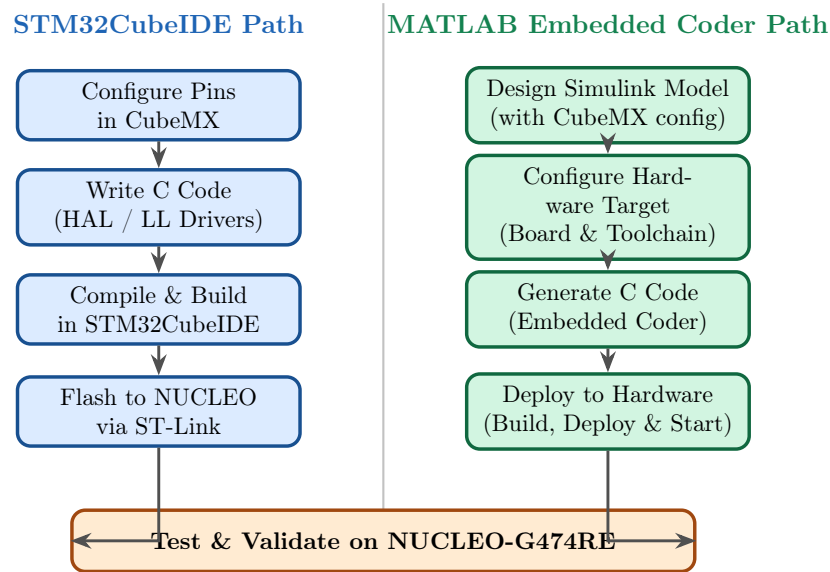


Figure 2: Dual-framework development paths: STM32CubeIDE (blue, left) follows manual C firmware development whilst MATLAB Embedded Coder (green, right) uses model-based code generation. Both paths converge at the same NUCLEO-G474RE target for validation.

[SYSTEM ARCHITECTURE PNG]

Replace with:

```
\includegraphics[width=0.85\textwidth]{NN_Folder/system_architecture.png}
```

Use for: overall system block diagram, wiring diagram, or any high-level architectural overview image. Check the below image for an example.

SYSTEM ARCHITECTURE

Embedded Hardware-Software Interaction: Timer-Triggered ADC Sampling, NVIC Interrupt Handling, and Superloop Code Execution

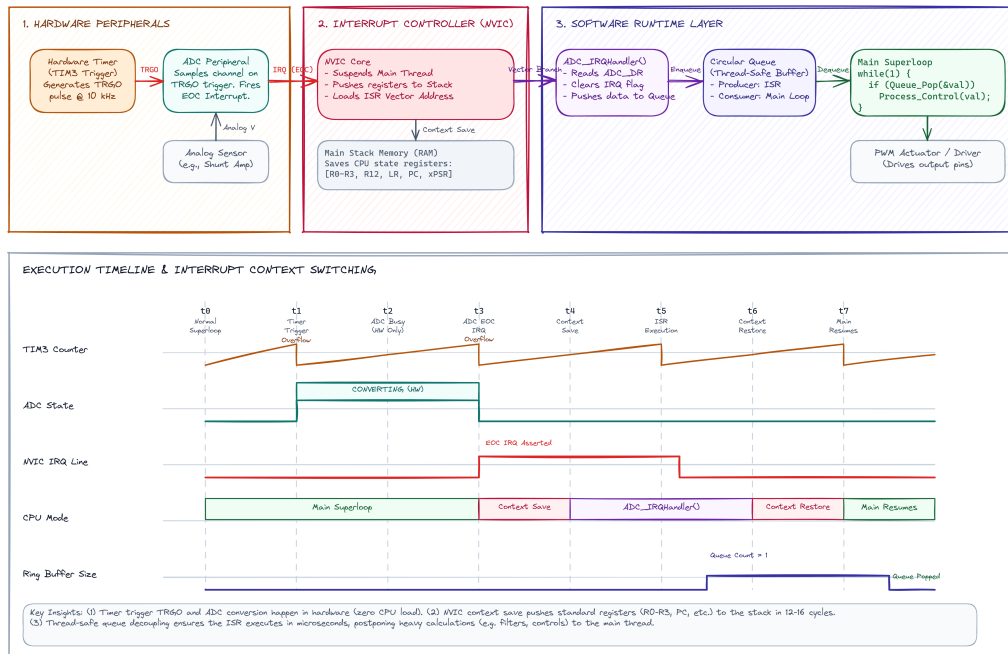


Figure 3: [Guidance: Write your caption here, max 45 words. Describe the main components shown, their interconnections, and the direction of data flow. Do not repeat what is already stated in the surrounding paragraph text.]

2 STM32CubeIDE Implementation

2.1 Prerequisites

Before starting this implementation, ensure you have completed the previous projects in this series, available at the project repository.

- STM32CubeIDE version 1.10 or later installed
- NUCLEO-G474RE development board and USB cable
- Completion of Projects 01–NN-1 for foundational knowledge
- [Guidance: Add any project-specific prerequisites, e.g. “Oscilloscope or logic analyser for PWM verification.”]

2.2 Hardware Configuration

2.2.1 Pin Configuration

Table 1: Pin Configuration

Pin	Function	Description
PA5	GPIO Output	On-board green LED (LD2)
PC13	GPIO Input	On-board user button (B1), falling-edge trigger
PXX	Function	<i>[Guidance: Add your pin, e.g. “ADC1 CH1 — analogue input from potentiometer on PA0.”]</i>

Note: Refer to the STM32G474RE Reference Manual (RM0440) for complete pin definitions and electrical specifications. The NUCLEO-G474RE User Manual (UM2505) documents morpho and Arduino connector pinouts.

2.2.2 Peripheral Configuration

Timer (TIMx):

Table 2: TIMx Configuration — *[state purpose, e.g. “20 kHz PWM”]*

Parameter	Value and Rationale
Clock Source	Internal (170 MHz system clock)
Prescaler (PSC)	<i>[value]</i> → Timer clock = 170 MHz / (PSC+1)
Auto-Reload (ARR)	<i>[value]</i> → Period = ARR ticks / timer clock
Mode	<i>[Up / Centre-aligned / etc.]</i>
Interrupt / DMA	<i>[Enabled / Disabled and reason]</i>

Other Peripherals (ADC / DAC / UART / HRTIM):

[Guidance: For each additional peripheral, include a similar table or a short paragraph describing resolution, sampling rate, DMA channels, and any relevant clock-domain notes. Reference the matching CubeMX screenshot in the MATLAB section or Appendix.]

2.3 Software Architecture

2.3.1 Project Structure

```

1 NN_ProjectName/
2 |-- Core/
3 |   |-- Inc/
4 |   |   |-- main.h
5 |   |   '-- stm32g4xx_it.h
6 |   '-- Src/
7 |       |-- main.c           % Entry point; calls MX_xxx_Init()
8 |       |-- stm32g4xx_it.c   % ISR handlers (HAL callbacks here)
9 |       '-- <your_module>.c  % Project-specific logic
10 |-- Drivers/

```



```

11 |   '-- STM32G4xx_HAL_Driver/
12 '-- <ProjectName>.ioc           % CubeMX config (open to regenerate)

```

Listing 1: STM32CubeIDE project directory layout

2.3.2 Initialisation Flow

[Guidance: Briefly describe the sequence of `MX_xxx_Init()` calls generated by CubeMX and any custom setup performed after them. One short paragraph is sufficient; the code listing below carries the detail.]

2.4 Code Implementation

2.4.1 Peripheral Initialisation

```

1 /* USER CODE BEGIN 2 */
2 // Start TIMx in interrupt mode
3 HAL_TIM_Base_Start_IT(&htimX);
4
5 // Start ADC with DMA (if applicable)
6 HAL_ADC_Start_DMA(&hadcX, (uint32_t*)adc_buf, BUF_LEN);
7 /* USER CODE END 2 */

```

Listing 2: Key peripheral initialisation (auto-generated by CubeMX)

2.4.2 Core Algorithm / Control Law

```

1 // [Guidance: Paste the key function here.
2 // Keep ISRs short; offload heavy computation to main loop via flags
3 // .]
4 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
5 {
6     if (htim->Instance == TIM1)
7     {
8         // Time-critical action here (keep < 10 us for 20 kHz loops)
9     }
10 }

```

Listing 3: Core processing function

2.5 Building and Flashing

1. Open the `.ioc` file in STM32CubeMX; verify pin and peripheral settings match the tables above.
2. Right-click project → **Build Project**. Resolve any compiler errors before proceeding.
3. Connect the NUCLEO board via USB (ST-Link port).

4. Right-click project → **Run As** → **STM32 C/C++ Application**.

Note: If the board is not detected, install ST-Link USB drivers from STMicroelectronics and try a different USB port.

3 MATLAB Embedded Coder Implementation

3.1 Prerequisites

Before starting this implementation, ensure you have completed the previous projects at the project repository.

- *MATLAB R2025a or later with Embedded Coder toolbox*
- *Embedded Coder Support Package for STMicroelectronics STM32 (install via Add-On Explorer)*
- *ARM GCC Compiler configured in MATLAB — change from default to **GNU Tools for ARM Embedded Processors***
- *NUCLEO-G474RE development board and USB cable*
- *[Add any project-specific prerequisites here.]*

Note: The ARM GCC compiler selection is critical. By default, MATLAB uses a different toolchain; this must be changed to **GNU Tools for ARM Embedded Processors** before the first build attempt.

3.2 MATLAB/Simulink Model Design

3.2.1 Model Architecture

[Guidance: Describe the top-level structure of your Simulink model. List the main subsystems and their roles. State the sample time and solver settings. For co-simulation projects (see Project 06), explain the two-model split here.]

- **Input / Measurement subsystem:** *[e.g. ADC read, signal scaling, sanity filtering.]*
- **Control Algorithm subsystem:** *[e.g. PI controller, feedforward, gain scheduling.]*
- **Output / Actuation subsystem:** *[e.g. PWM write, DAC output, UART transmit.]*

3.2.2 Subsystem Summary

Table 3: Simulink subsystem descriptions

Subsystem / Block	Purpose
<i>Subsystem name</i>	<i>[Guidance: One sentence describing what this subsystem does and why it is a separate block.]</i>
<i>MATLAB Function: xxx</i>	<i>[Guidance: Describe the codegen function; reference the Appendix listing, e.g. “Implements the PI control law with anti-windup; see Appendix A.”]</i>
<i>ISR Block / Triggered Subsystem</i>	<i>[Guidance: State which TIM or event triggers it and the sample period.]</i>

3.3 Model Configuration

3.3.1 Solver Settings

- **Solver Type:** Fixed-step
- **Solver:** Discrete (no continuous states) — use *ode3 (Bogacki-Shampine)* only for Simscape models
- **Fixed Step Size:** *[e.g. 1/20000 for a 20 kHz control loop]*

3.3.2 Hardware Target Settings

[Guidance: Confirm the following in Simulink Configuration Parameters → Hardware Implementation: Board: STM32G4xx based; Toolchain: GNU Tools for ARM Embedded Processors; External mode: off (unless using serial monitoring).]

3.4 Code Generation and Deployment

1. Press **Ctrl+E** to open Configuration Parameters. Verify board and toolchain settings.
2. Connect the NUCLEO board via USB.
3. In the Simulink toolbar, click **Deploy to Hardware** (Build, Deploy & Start).
4. MATLAB generates C code, compiles it with ARM GCC, and flashes the resulting binary via ST-Link.

Note: Generated source files are placed in a folder named `<ModelName>_ert_rtw/` next to the `.slx` file. These are auto-generated; do not edit them directly.

3.5 MATLAB Functions (Codegen)

[Guidance: List each MATLAB Function block used (e.g. `buck_controller.m`, `parse_rx.m`), state its inputs/outputs, and cross-reference its Appendix listing. Example: “The control law is implemented in `buck_controller` (Appendix A). It accepts V_{ref} , V_o , and V_{in} ; returns duty ratio $D \in [0, 0.95]$.”]

4 Testing and Validation

4.1 Hardware Testing

Before running the full system, verify each physical element:

1. *[Guidance: Step 1 — verify power and connectivity, e.g. “Confirm the board enumerates as a COM port.”]*
2. *[Guidance: Step 2 — verify a key peripheral in isolation, e.g. “Toggle PA5 manually in CubeIDE Live Expressions to confirm LED responds.”]*

3. *[Guidance: Step 3 — verify signal chain, e.g. “Probe PA8 with an oscilloscope; confirm 20 kHz PWM at the expected duty cycle.”]*

4.2 Model Simulation Testing

Before deploying to hardware, validate in Simulink simulation mode:

- Replace hardware I/O blocks with step sources and scopes.
- Apply a step reference and verify the controller converges to setpoint within the expected settling time.
- Check that all saturation blocks activate only within their intended operating ranges.

4.3 Integration Testing

[Guidance: Describe 3–5 checks that confirm the full hardware-software loop is working correctly. Include any checks specific to your peripheral combination, e.g. DMA buffer integrity, UART packet format, or ADC/DAC linearity.]

5 Results and Analysis

5.1 Experimental Results

[Guidance: Include at minimum (a) a time-domain waveform showing system response, (b) a steady-state result confirming accuracy, and (c) optionally a PWM or hardware signal verification. Follow the 45-word caption rule strictly.]

Table 4: Summary of experimental results

Measurement	Value / Observation
Parameter 1	Result
Parameter 2	Result
Parameter 3	Result

5.2 Key Observations

[Guidance: 3–5 bullet points summarising the most significant findings. Connect each observation back to a specific result image or table above. End with one sentence on any discrepancy between simulated and measured behaviour.]

6 Troubleshooting

Table 5: Common issues and solutions

Issue	Solution
Code does not compile	Check that the correct board is selected in CubeMX and that all generated files are included in the build path
Board not detected	Install ST-Link drivers; try a different USB port; restart the IDE
MATLAB deploy fails	Confirm GNU Tools for ARM Embedded Processors is selected as the active toolchain
<i>[Project-specific issue]</i>	<i>[Concise, actionable solution.]</i>
<i>[Project-specific issue]</i>	<i>[Concise, actionable solution.]</i>

7 Extensions and Enhancements

7.1 Possible Improvements

1. **[Enhancement 1 title]:** *[Guidance: Describe a concrete, implementable improvement. State what changes in hardware or software and what additional capability it provides. Close with one sentence on how to start.]*
2. **[Enhancement 2 title]:** *[Guidance: Same structure.]*
3. **[Enhancement 3 title]:** *[Guidance: Same structure.]*

7.2 Advanced Topics

- **FreeRTOS integration:** Replace the bare-metal superloop with a real-time OS to manage multiple concurrent control tasks with deterministic scheduling.
- **Multi-phase / multi-level topologies:** Extend the PWM and ADC configuration to support interleaved or cascaded converter stages.
- *[Guidance: Add 1–2 advanced topics specific to your project domain.]*

8 Conclusion

[Guidance: Three to four sentences. State (1) what was demonstrated, (2) which technique or peripheral was the key contribution, (3) the primary limitation or open question, and (4) how this project builds toward the next one in the series. Do NOT say “this project successfully demonstrated” — start with the actual result.]

Key Takeaways:

- *[Takeaway 1: One sentence, specific to this project.]*
- *[Takeaway 2: One sentence.]*
- *[Takeaway 3: One sentence on tool selection or framework trade-off learned.]*

References

- [1] STMicroelectronics, *STM32G4 Series Advanced ARM-based 32-bit MCUs Reference Manual*, RM0440.
- [2] STMicroelectronics, *STM32G474RE Datasheet*.
- [3] STMicroelectronics, *STM32G4 NUCLEO-64 Boards User Manual*, UM2505.
- [4] STMicroelectronics, *Description of STM32G4 HAL and Low-Layer Drivers*, UM2570.
- [5] MathWorks, *Embedded Coder Documentation*.
- [6] MathWorks, *Embedded Coder Support Package for STMicroelectronics STM32*.
- [7] STMicroelectronics, *HRTIM Cookbook*, AN4539.

A Code Listings

[Guidance: Place full MATLAB function code, helper utilities, or any code too long for the main body here. Use a separate \subsection per function. Use the [language=Matlab] or [language=C] option in \begin{lstlisting}.]

```
1 function output = your_function(input1, input2)
2 %#codegen
3 % Brief description: what this computes and why.
4 % Inputs:   input1 - description [units / range]
5 %           input2 - description [units / range]
6 % Outputs:  output - description [units / range]
7
8 % Implementation here
9 output = input1 + input2;
10 end
```

Listing 4: your_function.m — brief one-line description

B Peripheral Configuration Screenshots

[Guidance: Move any CubeMX screenshots that would interrupt the flow of the main text here. Use the paired \subfigure pattern at 0.48\textwidth each.]

C Hardware Documentation

Refer to the official NUCLEO-G474RE schematic and the STM32G474RE Reference Manual (RM0440) for complete hardware specifications, electrical characteristics, and pin definitions.

Author

Anmol Govindarajapuram Krishnan

GitHub: <https://github.com/Anmol-G-K>

Email: cb.en.u4eee23103@cb.students.amrita.edu

Acknowledgments

- **STMicroelectronics** for comprehensive MCU documentation and development tools
- **MathWorks** for MATLAB and Embedded Coder resources
- The embedded systems and power electronics community for knowledge sharing and innovation
- Contributors and reviewers providing feedback on this project series

This project is licensed under the MIT License.

For updates and additional resources:

<https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>